VGG16_HOG_GIST_SVM_Kod_Aciklamasi.pdf - local repository text copy

VGG16 FROM SCRATCH, HOG, GIST AND RBF-SVM

CODE DESIGN AND IMPLEMENTATION REPORT

Combined Eye-Region Deepfake Detection Pipeline

Classes, functions, epochs, training logic, feature fusion, model selection, checkpoints and reproducibility controls

Source notebook: Goz_Bolgesi_VGG16_Sifirdan_HOG_GIST_RBF_SVM (1).ipynb

Prepared: 6 August 2026

1. Report Purpose and Scope

This report explains the uploaded notebook as an end-to-end machine-learning system. It does not merely restate the code. It describes why the major classes and functions were created, how each stage protects experimental validity, why the VGG16 network is trained across epochs, why handcrafted HOG and GIST descriptors are added, and why the final decision is delegated to a PCA-reduced RBF-SVM classifier.

The notebook follows a two-stage design. In the first stage, a VGG16 convolutional network is trained from random initialization on combined-eye ROI images. In the second stage, the learned 512-dimensional VGG representation is fused with 3,780-dimensional HOG and 512-dimensional GIST-style Gabor features. Each block is standardized using training-only statistics, concatenated, reduced through PCA and classified with an RBF-SVM.

2. Overall Architecture and Design Rationale

Stage	Role	Why it is needed
Combined-eye ROI input	Uses both eyes in one image.	Preserves bilateral symmetry, eyelid/iris texture and cross-eye appearance inconsistencies.
Scratch VGG16	Learns task-specific deep visual features.	Tests whether the dataset alone can teach useful eye-region representations without ImageNet transfer.
HOG	Encodes local gradient orientations.	Captures edges, eyelid contours, iris boundaries and structural artifacts.
GIST-style Gabor	Encodes multi-scale and multi-orientation responses.	Summarizes global orientation-frequency structure and texture.
Feature standardization	Normalizes each feature block.	Prevents large-scale dimensions from dominating PCA and SVM.
PCA	Reduces the fused representation to at most 256 dimensions.	Controls dimensionality, noise, memory and RBF-SVM complexity.
RBF-SVM	Creates a nonlinear decision boundary.	Models interactions among heterogeneous deep and handcrafted features.
Validation threshold	Chooses the decision cutoff by validation F1.	Avoids assuming that the default SVM boundary is optimal for the

Stage	Role	Why it is needed
		selected metric.

3. ExperimentConfig Class: Why a Configuration Class Was Written

ExperimentConfig is a dataclass that centralizes paths, model parameters, augmentation settings, handcrafted-
feature settings, PCA and SVM search values. Without this class, experimental values would be scattered across
cells, making the notebook harder to audit, reproduce and modify. The dataclass also allows the final resolved
configuration to be serialized into the run folder.

Parameter	Value	Technical explanation
seed	42	Fixes random behavior as far as the framework permits, enabling repeatable splits, shuffling and initialization.
image_size	224	Standard square size used by VGG16 while keeping GPU memory manageable.
batch_size	16	A compromise between gradient stability and Colab memory usage for a fully trainable VGG16.
epochs	30	Maximum number of complete passes over the training data; early stopping can end training sooner.
early_stopping_patience	7	Stops when validation F1 fails to improve for seven consecutive epochs, reducing wasted computation and overfitting.
learning_rate	1e-4	A cautious rate because every VGG16 layer starts randomly and is trainable.
weight_decay	1e-4	AdamW regularization that discourages excessively large weights.
dropout	0.40	Strong regularization in the compact classifier to reduce co-adaptation.
hidden_dim	256	Intermediate classifier capacity after the 512-dimensional pooled VGG vector.
HOG size	128 x 64	Preserves the approximately two-eye horizontal layout while controlling descriptor size.
PCA components	256	Upper bound for the fused vector after training-only dimensionality reduction.
SVM C	0.1, 1, 10, 100	Tests different penalties for training errors and margin width.
SVM gamma	scale, 1e-3, 1e-4	Tests different RBF influence radii

Parameter	Value	Technical explanation
and nonlinear complexity.		

4. Reproducibility and Safe File-Writing Functions

Function	Purpose and reason
set_global_seed	Seeds Python, NumPy and PyTorch; requests deterministic cuDNN behavior so repeated runs are comparable.

atomic_write_text / json / csv
Writes to a temporary file and replaces the target only

after completion, reducing corruption after Colab or
local source interruptions.
atomic_save_checkpoint
Saves model state, reloads the temporary file and
verifies required keys before replacing the final
checkpoint.
atomic_joblib_dump / atomic_save_npz Safely stores sklearn objects and numerical feature
artifacts.
sha256_file Creates file hashes for split-leakage checks and
output integrity.
save_figure Saves and closes plots consistently, avoiding memory
accumulation.
clean_for_yaml Converts complex Python values into serializable
configuration values.
Atomic writing is particularly important in local source package workflows. A runtime can disconnect
while a file is being
written; temporary-file replacement ensures that an existing valid artifact is not silently replaced by
an incomplete
one.
5. Metadata Resolution, Cleaning and Data-Leakage Controls
The metadata stage accepts only successful combined-eye samples, resolves file paths, creates or
validates
sample identifiers, checks source video identifiers and calculates image hashes. The most important
methodological rule is source-video isolation: frames from one video must not appear in more than one
split.
Otherwise the model could recognize video-specific compression, identity or capture conditions rather
than
general deepfake evidence.
☐ Mandatory columns are checked before training.
☐ Rows with missing or invalid combined-eye images are rejected.
☐ Duplicate sample IDs are blocked.
☐ The same SHA-256 image hash is not allowed across train, validation and test.
☐ Missing video IDs are blocked.
☐ Source videos are asserted to belong to only one split.
☐ Test data remains untouched during CNN and SVM selection.

6. Letterbox and EyeCombinedDataset Classes
6.1 Letterbox class
Letterbox uses ImageOps.pad to resize the image without stretching its anatomy. A direct square resize
could
distort the horizontal eye layout, iris shape or eyelid geometry. Padding preserves aspect ratio and
centers the
ROI in a 224 x 224 canvas.
6.2 Why EyeCombinedDataset was written
A custom PyTorch Dataset is required because each metadata row must be converted into a synchronized
image tensor, integer target and traceability record. The class hides file-reading details from the
training loop and
guarantees a standard dictionary output.
Method / return field Role
__init__ Stores a clean dataframe copy and the selected
transform.
__len__ Reports dataset size to DataLoader.
__getitem__ Loads one image, converts it to RGB, applies
transforms and returns image/target/metadata.
sample_id Links predictions to the exact ROI sample.
source_video Enables later video-level aggregation and leakage
audits.
path Preserves the resolved source path for debugging.
7. Train-Only Normalization and Mild Augmentation
RGB mean and standard deviation are calculated only from the training split. Validation and test are
normalized
with those same values. This prevents held-out image distributions from influencing preprocessing
parameters.
Transformation Reason
Horizontal flip, p=0.50 Adds left-right variation while generally preserving

real/fake semantics.
Random affine, ±5 degrees Models small alignment errors without destroying eye anatomy.

Translation, 3% Improves tolerance to slight ROI localization shifts.
Brightness/contrast jitter, 0.08 Models modest illumination and compression variation.
No augmentation in validation/test Keeps evaluation deterministic and comparable.

8. VGG16ScratchClassifier Class

VGG16ScratchClassifier calls torchvision.models.vgg16(weights=None). The weights=None argument is decisive: no pretrained parameters are downloaded or loaded. Therefore all convolutional filters must be learned from the eye ROI dataset.

Component Function and rationale
network.features Retains the standard VGG16 convolution and pooling stack.
AdaptiveAvgPool2d((1,1)) Converts the final 512-channel feature map to one value per channel, independent of spatial size.
Linear 512→256 Creates a compact task-specific hidden representation.
ReLU Introduces nonlinearity.
Dropout 0.40 Regularizes the classifier.
Linear 256→2 Produces logits for real and fake classes.
extract_features Exposes the 512-dimensional learned descriptor for later HOG/GIST fusion.
forward Runs feature extraction and classification during CNN training.

```
network = vgg16(weights=None)
self.features = network.features
self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
self.classifier = nn.Sequential(
    nn.Linear(512, hidden_dim), nn.ReLU(inplace=True),
    nn.Dropout(p=dropout), nn.Linear(hidden_dim, 2)
)
```

The original large VGG16 ImageNet classifier is intentionally replaced. A compact head is more suitable for a smaller two-class eye dataset and substantially reduces classifier parameters.

9. Why Epochs Are Used

One epoch means that the optimizer has processed every training sample once. A network trained from random initialization cannot learn all useful edge, texture and class patterns in one pass. Repeated epochs allow gradients from many mini-batches to gradually update early edge filters, middle-level texture/shape filters and late task-specific representations.

Concept Meaning in this notebook
Maximum 30 epochs Provides enough opportunity for scratch training but is not a requirement to use all 30.
Batch size 16 Weights are updated after each mini-batch, so one epoch contains many optimization steps.
Validation after every epoch Measures generalization on unseen validation data.
Best validation F1 Determines which checkpoint is considered best.
Patience 7 Ends training when no better validation F1 occurs for seven epochs.

In the recorded run, the best validation F1 of approximately 0.6453 first appeared at epoch 4. Epochs 5-11 did not exceed it, so early stopping activated after seven non-improving epochs. Thus, 30 was only the upper limit; training ended at epoch 11. This behavior prevents unnecessary computation and reduces the chance of continuing to memorize training noise.

10. Class Weights, Loss, AdamW and Scheduler

10.1 compute_class_weights and CrossEntropyLoss

compute_class_weights calculates inverse-frequency weights from the training labels. CrossEntropyLoss then

penalizes mistakes on the less frequent class more strongly. This is intended to reduce majority-class bias
without changing validation or test distributions.
10.2 AdamW optimizer

AdamW is used because it adapts the learning rate for each parameter and applies decoupled weight decay. The 1e-4 learning rate is conservative for a large scratch network, while weight decay adds regularization.
10.3 Learning-rate scheduler
The training cell uses a validation-based scheduler state stored in every checkpoint. Its role is to lower the
effective learning rate when progress stalls, allowing smaller updates near a potentially useful solution. Saving
scheduler_state_dict is necessary for faithful resume behavior.
11. Quality Gates Before Full Training
run_quality_gates performs two-batch forward/backward testing before the expensive run. It checks finite losses,
finite gradients, optimizer updates, checkpoint save/load and output equivalence in a newly constructed model.
This catches broken shapes, invalid data, NaN/Inf propagation and incomplete checkpoint logic early.
Gate Failure it prevents
Two-batch smoke test Long runs that fail immediately because of shape or device errors.
Finite-loss check Training on NaN/Inf values.
Finite-gradient check Corrupted parameter updates.
Atomic checkpoint test Unrecoverable or partially written checkpoints.
Fresh-model equivalence Checkpoints that cannot reproduce the saved model outputs.
12. binary_metrics and run_cnn_epoch
12.1 binary_metrics
binary_metrics computes accuracy, precision, recall, F1, specificity, ROC-AUC and average precision. Specificity
is particularly important here because the final model shows high fake recall but low real-class recognition.
Reporting only accuracy or recall would hide this asymmetry.
12.2 run_cnn_epoch
run_cnn_epoch encapsulates one complete training or validation pass. In training mode it clears gradients,
performs forward propagation, computes loss, backpropagates, updates parameters and records predictions. In
evaluation mode it performs the same inference and metrics logic without parameter updates. Sharing one function minimizes train/validation inconsistencies.
13. Checkpoint Strategy and Resume Safety
Artifact Meaning
best.ckpt Checkpoint with the highest validation F1.

Artifact Meaning
last.ckpt Most recently completed epoch, useful for resuming after interruption.
epoch_N.ckpt Recent historical checkpoints for debugging or rollback.
checkpoint rotation Keeps only the most recent three epoch snapshots to control storage usage.
The checkpoint contains epoch, model, optimizer, scheduler, best score, configuration and Python/NumPy/PyTorch random states. Saving all of these is necessary because model weights alone cannot reproduce the exact continuation of an interrupted run.
14. Fresh-Load Inference Test
After training, best.ckpt is loaded into a newly constructed VGG16ScratchClassifier. A validation batch must
produce a finite tensor with two output columns. Feature extraction is allowed only after this test passes. This
protects the second stage from silently using a corrupted, incompatible or incompletely loaded CNN.
15. HOG Feature Extraction
HOG represents the distribution of local gradient directions. The image is letterboxed to 128 x 64, converted into
gradient cells and summarized into normalized blocks. With 9 orientations, 8 x 8 pixel cells and 2 x 2 cell blocks,
the notebook produces 3,780 HOG values per sample.

Setting	Why
128 x 64	Retains a wide combined-eye layout and limits computation.
9 orientations	Provides a useful angular resolution for edges and contours.
8 x 8 pixels per cell	Captures local structures such as eyelid and iris boundaries.
2 x 2 cells per block	Normalizes neighboring cells and improves illumination tolerance.

16. GIST-Style Gabor Feature Extraction

The GIST-style descriptor applies Gabor filters at four scales and eight orientations. Each response is pooled over a 4 x 4 spatial grid, producing 4 × 8 × 16 = 512 values. This summarizes where different orientation-frequency patterns occur across the two-eye ROI.

The implementation is described as GIST-style rather than a canonical external GIST package because it builds a deterministic Gabor-bank and grid-pooling descriptor directly in the notebook.

17. Learned VGG Feature Extraction and L2 Normalization

extract_vgg_feature_from_path loads the original image with the deterministic evaluation transform and passes it through feature_model.extract_features. The output is the 512-dimensional global-pooled VGG representation learned during scratch CNN training. HOG, GIST and VGG vectors are L2-normalized to reduce sensitivity to absolute magnitude before block standardization.

18. Why Feature Extraction Uses Original Images Without Training Augmentation

The SVM stage requires one stable feature vector per sample. Therefore HOG, GIST and VGG representations are extracted deterministically from the original ROI rather than random augmented views. This ensures that cached or saved features remain reproducible and that validation comparisons are not affected by random transforms.

19. Train-Only StandardScaler and Feature Fusion

Separate StandardScaler objects are fitted for HOG, GIST and VGG blocks using only training samples. Separate fitting is important because the three descriptors have different distributions. The transformed blocks are then concatenated into a 4,804-dimensional fused vector: 3,780 HOG + 512 GIST + 512 VGG. Validation and test use scaler.transform only. Calling fit on held-out splits would leak their distribution into model development.

20. PCA: Why 4,804 Features Are Reduced

The fused vector is high-dimensional relative to the number of training samples. PCA is fitted on training data and limited to 256 components, or fewer if sample/dimension constraints require it. The reduction suppresses redundancy, decreases SVM memory and runtime, and mitigates overfitting caused by sparse high-dimensional geometry.

21. RBF-SVM and Hyperparameter Search

21.1 Why an SVM is used after VGG16

The CNN is used as a learned feature extractor, while the SVM performs the final classification on the fused descriptor. This hybrid design allows handcrafted and learned evidence to interact in a classical nonlinear classifier and provides an explicit ablation-friendly representation.

21.2 Meaning of C and gamma

Parameter	Meaning
C	Controls the penalty for training errors. Small C favors a wider smoother margin; large C fits training points more strongly.
gamma	Controls how quickly the RBF similarity decreases with distance. Large gamma creates local complex boundaries; small gamma creates smoother global boundaries.

class_weight=balanced Adjusts class influence according to training
frequencies.
decision_function Produces continuous signed scores used for
threshold optimization and ROC/PR analysis.
21.3 Why validation F1 selects the SVM and threshold

Every C/gamma candidate is evaluated on validation data. The code also searches a decision threshold that

maximizes validation F1 instead of assuming zero is optimal. The test split is not loaded into this
selection stage,
preserving an unbiased final evaluation.

22. Final Test Evaluation and Video Aggregation
Only after CNN checkpoint, SVM hyperparameters and threshold are fixed are test features extracted.
Frame-
level predictions are saved with sample metadata. Scores are then aggregated by source video to measure
whether combining multiple frames produces a more stable decision.
Level Accuracy Precision Recall F1 Specificity ROC-AUC
Frame 0.5596 0.5437 0.9167 0.6826 0.1781 0.6226
Video 0.5000 0.5000 1.0000 0.6667 0.0000 1.0000
The frame result is strongly biased toward the fake class: recall is high, but specificity is very low.
The video-level
ROC-AUC of 1.0 must be interpreted cautiously because the reported video sample count may be very small;
accuracy and specificity show that the selected threshold classified all videos as fake. Therefore this
run should
not be presented as a balanced detector.
23. Plotting Functions and Why Each Figure Is Produced
Figure / function Interpretation
CNN loss curve Shows optimization progress and whether validation
loss diverges from training loss.
CNN accuracy/F1 curves Shows class decision performance across epochs.
plot_confusion_matrix Reveals false-positive and false-negative counts.
plot_roc Measures score ranking independent of one
threshold.
plot_precision_recall Emphasizes positive-class performance, useful under
imbalance.
Decision-score histogram Shows overlap between real/fake score distributions
and the selected threshold.
Dataset distribution Audits class/split counts before training.
24. Final Output Audit and Reproducibility Package
The final cell captures pip freeze, environment versions, hardware/device details, figure dimensions,
file hashes,
run summary and a complete output manifest. This transforms the notebook from a one-off experiment into
an
auditable run package.
☐ requirements_lock.txt records exact installed packages.
☐ environment.json records Python, PyTorch, torchvision, OpenCV, NumPy, pandas and device information.
☐ figure_quality_audit verifies minimum image resolution.
☐ output_manifest.csv lists generated artifacts and hashes.
☐ run_summary.json gathers final metrics and selected settings.
25. Technical Findings and Improvement Recommendations
Finding Impact Recommended action
Scratch VGG16 loss remains near
0.693
The CNN learned little beyond
chance-level cross-entropy
behavior.
Use longer but better-controlled
training, stronger normalization
checks, learning-rate experiments
or transfer learning as a
comparison.

Finding Impact Recommended action
Validation F1 jumps between 0
and ~0.645
Predictions are unstable and may
collapse toward one class.

Inspect confusion matrices per epoch, logits and class-weight effects; select checkpoint using ROC-AUC plus balanced metrics.
Final specificity is 0.178 Many real samples are classified as fake.
Optimize a balanced objective, report balanced accuracy/MCC, calibrate threshold and examine false positives.
Video threshold predicts all videos fake
Video-level accuracy/F1 can be misleading.
Increase independent test videos and select aggregation/threshold only on validation videos.
PCA + RBF-SVM uses a large fused vector
Can overfit when sample count is limited.
Compare HOG-only, GIST-only, VGG-only and pairwise ablations; use nested or grouped validation where feasible.
Duplicate helper definitions exist
Later cells overwrite earlier function names, increasing maintenance risk.
Keep one canonical implementation for each atomic writer and hash function.
Deprecation warnings appear in notebook output
Does not invalidate results but creates noisy logs.
Update libraries/runtime or filter only known harmless warnings.
No pretrained baseline in this notebook
Scratch result alone cannot establish whether the architecture is unsuitable.
Compare against pretrained VGG16/Swin/EfficientNet under identical split and metrics.

26. Cell-by-Cell Notebook Map

Cell(s)	Role
0	Experiment objective, combined-eye rationale and reliability rules.
1	Installs missing packages.
2	Imports libraries and mounts local source package.
3	Defines ExperimentConfig, validates paths and creates run folders.
4	Defines seeds, atomic writers, checkpoint helpers and figure utilities.
5-7	Loads/cleans metadata, checks leakage and plots distribution.
8-10	Computes train-only RGB statistics and constructs transforms/loaders.
11-12	Defines and instantiates scratch VGG16 classifier.
13-14	Runs mandatory smoke, numerical and checkpoint quality gates.
15-17	Trains CNN with early stopping and saves learning curves.
18-19	Fresh-loads best CNN and validates inference.

Cell(s)	Role
20-21	Defines and extracts HOG, GIST and learned VGG features.
22-23	Fits train-only scalers/PCA and selects RBF-SVM by validation F1.
24-25	Extracts test features and performs frame/video evaluation.
26-28	Creates confusion matrix, ROC, PR and score-distribution figures.
29-30	Audits figures, captures environment and writes final manifest.
31	Documents expected output folder structure.

27. Conclusion

The notebook is a carefully audited hybrid experiment rather than a simple VGG16 training script. Its strongest software qualities are source-video isolation, training-only preprocessing, atomic artifacts, checkpoint integrity tests, fresh-load inference validation and strict test isolation. The architecture explicitly combines structural gradients, global Gabor texture and learned CNN representations before nonlinear SVM classification.

The recorded results, however, show that the scratch VGG16 branch did not learn a stable balanced representation: loss stayed close to random binary cross-entropy, validation F1 was unstable and final specificity was low. Consequently, the code framework is methodologically useful, but the model result should be treated as a baseline that requires stronger representation learning, ablation analysis and more robust video-level validation.